

# **STUDENT ATTENDANCE PORTAL**

## **TUTORIAL 2 IN**

### **24CSC42 FULLSTACK DEVELOPMENT**

**Submitted by**

**NAME: VISHNUPREETHI B**

**ROLL NUMBER: 24CSR350**

*in partial fulfilment of the  
requirements for the award of the  
degree of*

## **BACHELOR OF ENGINEERING**

**IN**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**



**PERUNDURAI ERODE-638060**

**APRIL 2026**

## SCENARIO

The Student Attendance Portal is a web-based application designed to simplify and digitize the process of managing student attendance in educational institutions. Traditionally, attendance is recorded manually, which can lead to errors, data loss, and difficulty in tracking records over time. This application provides an efficient and user-friendly solution by automating attendance management using modern web technologies.

The system is developed using React.js for the frontend, Node.js with Express.js for the backend, and MongoDB as the database to store attendance records. It follows a RESTful architecture, enabling smooth communication between the client and server.

In this portal, users can view a list of students along with their attendance status (Present or Absent). They can update attendance with a single click and remove incorrect entries using the delete option. All these operations are performed dynamically, so the user interface updates instantly without requiring a page refresh, improving the overall user experience.

The frontend uses React functional components and Axios to send HTTP requests (GET, PUT, DELETE) to the backend API, while the backend processes these requests and interacts with the MongoDB database to manage the records.

Overall, the Student Attendance Portal demonstrates the integration of frontend and backend technologies to build a responsive and real-time application, showcasing the implementation of CRUD operations using REST APIs.

## PROJECT STRUCTURE:

Tutorial2/

```
├── server/
│   ├── models/
│   ├── server.js
│   ├── package.json
│   └── package-lock.json
└── client/
    ├── public/
    ├── src/
    ├── index.html
    └── vite.config.js
├── package.json
└── package-lock.json
```

## Frontend

### APP.jsx

```
import React, { useState } from 'react'; import { BrowserRouter as Router, Routes,
Route, Navigate } from 'react-router-dom'; import Login from './components/Login';
import Register from './components/Register'; import AttendancePortal from
'./components/AttendancePortal'; import './App.css' function App() { const [user,
setUser] = useState(null); const handleLogin = (userData) => { setUser(userData);
};
const handleLogout = () => {
  setUser(null);
};
return (
  <Router future={{ v7_startTransition: true, v7_relativeSplatPath: true }}>
    <div className="App">
```

```

    <Routes>
      <Route path="/login" element={user ? <Navigate to="/attendance" /> : <Login
onLogin={handleLogin} />} />
      <Route path="/register" element={<Register />} />
      <Route path="/attendance" element={user ? <AttendancePortal user={user}
onLogout={handleLogout} /> : <Navigate to="/login" />} />
      <Route path="/" element={<Navigate to="/login" />} />
    </Routes>
  </div>
</Router>
);
}export default App;

```

### **main.jsx**

```

import React from 'react' import
ReactDOM from 'react-dom/client' import
App from './App.jsx' import './index.css'
ReactDOM.createRoot(document.getElementById('root')).render(
<React.StrictMode>
  <App />
</React.StrictMode>,)

```

### **api.js:**

```

const API_BASE_URL = 'http://localhost:5000/api'; export const
fetchAttendance = async () => {  const response = await
fetch(`${API_BASE_URL}/attendance`); if (!response.ok) {
throw new Error('Failed to fetch attendance records');
} return
response.json();
};

```

```

export const addAttendance = async (record) => {  const response
= await fetch(`${API_BASE_URL}/attendance`, {  method:
'POST',  headers: {
    'Content-Type': 'application/json',
  },
  body: JSON.stringify(record),
});
if (!response.ok) {  throw new Error('Failed to add
attendance record');
}
return response.json();
};

export const updateAttendance = async (id, status) => {  const response
= await fetch(`${API_BASE_URL}/attendance/${id}`, {
  method: 'PUT',
headers: {
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({ status }),
});if (!response.ok) {  throw new Error('Failed to
update attendance record');
}
return response.json();
};

export const deleteAttendance = async (id) => {  const response = await
fetch(`${API_BASE_URL}/attendance/${id}`, {  method: 'DELETE',
});
if (!response.ok) {  throw new Error('Failed to delete
attendance record');
}

```

```

    return response.json();
  };
  export const searchAttendance = async (query = "", status = 'All') => {
    const params = new URLSearchParams();
    if (query) params.append('search', query); if
(status !== 'All') params.append('status', status);
    const response = await fetch(`${API_BASE_URL}/attendance/search?${params.toString()}`);
    if (!response.ok) { throw new Error('Failed to search
attendance records');
    }
    return response.json();
  };

```

### **Addattendance.jsx**

```

import React, { useState } from 'react';
const AddAttendanceForm = ({ onAdd }) => {
  const [studentName, setStudentName] = useState("");
  const [rollNumber, setRollNumber] = useState("");
  const [date, setDate] = useState(""); const [status,
setStatus] = useState('Present'); const [error,
setError] = useState(""); const handleSubmit = (e) =>
{
  e.preventDefault();
  const trimmedName = studentName.trim();
  const trimmedRoll = rollNumber.trim(); if
(!trimmedName || !trimmedRoll || !date) {
    setError('Please fill in all fields before submitting.');
```

return;

```

  }

  if (!/^[0-9]+$/.test(trimmedRoll)) { setError('Roll
number must be a valid numeric value.');
```

```

        return;
    }
    setError("");
    onAdd({ studentName: trimmedName, rollNumber: trimmedRoll, date, status });
    setStudentName(""); setRollNumber("");
    setDate("");
    setStatus('Present');
};

return (
    <div className="add-attendance-form card">
        <h3>Add New Attendance Record</h3>
        <form onSubmit={handleSubmit}>
            <div className="form-group">
                <label>Student Name:</label>
                <input
type="text"
                value={studentName}      onChange={(e) =>
setStudentName(e.target.value)} placeholder="Enter student name />      </div>
                <div className="form-group">
                    <label>Roll Number:</label>
                    <input
                        type="text"
                        value={rollNumber}      onChange={(e) =>
setRollNumber(e.target.value)}
                    placeholder="Enter roll number"
                    />
                </div>
                <div className="form-group">
                    <label>Date:</label>

```

```

        <input          type="date"
value={date}          onChange={(e) =>
setDate(e.target.value)}
        />
    </div>
    <div className="form-group">
        <label>Status:</label>
        <select value={status} onChange={(e) => setStatus(e.target.value)}>
            <option value="Present">Present</option>
            <option value="Absent">Absent</option>
        </select>
    </div>
    {error && <p className="form-error">{error}</p>}
    <button type="submit" className="btn btn-success">Add Record</button>
</form></div>
);
}; export default AddAttendanceForm;

```

### **AddAttendanceList.css:**

```

import React, { useState } from 'react'; const AttendanceList =
({ records, onUpdate, onDelete }) => { const [editingId,
setEditingId] = useState(null);

    const [editStatus, setEditStatus] = useState(""); const
    handleEdit = (record) => {
        setEditingId(record.id);
        setEditStatus(record.status);
    };

    const handleSave = (id) => {
onUpdate(id, editStatus);
setEditingId(null);
    };

```



```

const handleCancel = () => {
  setEditingId(null);  setEditStatus("");
};

return (
  <div className="attendance-list card">
    <table>
      <thead>
        <tr><th>Student Name</th><th>Roll Number</th>
<th>Date</th><th>Status</th><th>Actions</th>
        </tr>
      </thead>
      <tbody>
        {records.map(record => (
          <tr key={record.id}>
            <td>{record.studentName}</td>
            <td>{record.rollNumber}</td>
            <td>{record.date}</td>
            <td>
              {editingId === record.id ? (
                <select
                  value={editStatus}          onChange={(e) =>
setEditStatus(e.target.value)}
                >
                  <option value="Present">Present</option>
                  <option value="Absent">Absent</option>
                </select>
              ) : (
                <span className={`status ${record.status.toLowerCase()}`}>{record.status}</span>
              )}
            </td><td>

```

```

      {editingId === record.id ? (
        <div>
          <button onClick={() => handleSave(record.id)} className="btn btn-small
btnsuccess">Save</button>
          <button onClick={handleCancel} className="btn btn-small
btnsecondary">Cancel</button> </> ) : (
            <button onClick={() => handleEdit(record)} className="btn btn-small
btnprimary">Edit</button>
            <button onClick={() => onDelete(record.id)} className="btn btn-small
btndanger">Delete</button></> )} </td></tr>
      )}
    </tbody>
  </table>
  {records.length === 0 && <p className="no-records">No attendance records found.</p>}
</div>
);
}; export default AttendanceList;

```

### **AttendancePortal.jsx:**

```

import React, { useState, useEffect } from 'react'; import
AttendanceList from './AttendanceList'; import
AddAttendanceForm from './AddAttendanceForm';
import { fetchAttendance, addAttendance,
updateAttendance, deleteAttendance, searchAttendance }
from '../services/api';

const AttendancePortal = ({ user, onLogout }) => { const
[attendanceRecords, setAttendanceRecords] = useState([]); const
[loading, setLoading] = useState(true); const [searchQuery,
setSearchQuery] = useState(""); const [statusFilter,
setStatusFilter] = useState('All'); useEffect(() => {
loadAttendance();
}, []);

```

```

const loadAttendance = async () => {
  try {
    const records = await fetchAttendance(); setAttendanceRecords(records);
  } catch (error) { console.error('Error fetching
attendance:', error);
  } finally { setLoading(false);
  } };

const handleAdd = async (newRecord) => {
  try {
    await addAttendance(newRecord);
    await loadAttendance(); // Refresh list
  } catch (error) { console.error('Error adding
attendance:', error); };
  const handleUpdate = async
(id, status) => {
    try {
      await updateAttendance(id, status);
      await loadAttendance(); // Refresh list
    } catch (error) { console.error('Error updating
attendance:', error);
    }
  };

  const handleDelete = async (id) => { if (window.confirm('Are you
sure you want to delete this record?')) {
    try {
      await deleteAttendance(id);
      await loadAttendance(); // Refresh list
    } catch (error) { console.error('Error
deleting attendance:', error);
    }
  }
}

```

```

};

useEffect(() => {    const applyFilter = async () => {    if
(searchQuery.trim() === " && statusFilter === 'All') {
await loadAttendance();
    } else {
try {
        const results = await searchAttendance(searchQuery, statusFilter);
setAttendanceRecords(results);
    } catch (error) {
console.error('Search error:', error);
await loadAttendance();
    } };;

    const debounceTimer = setTimeout(applyFilter, 300);
return () => clearTimeout(debounceTimer);
}, [searchQuery, statusFilter]);
return (

<div className="attendance-portal">
<header className="header">
    <div className="header-content">
        <h1>Students Attendance Portal</h1>
        <div className="user-info">
            <span>Welcome, Admin ({user.email})</span>
            <button onClick={onLogout} className="btn btn-secondary">Logout</button>
        </div>
    </div>
</header>
<main className="main-content">
    <div className="container attendance-grid">
        <div className="form-column">
            <AddAttendanceForm onAdd={handleAdd} />

```

```

</div>
<div className="attendance-section">
  <div className="records-header">
    <h2>Attendance Records</h2>
    <div className="search-panel">
      <input
type="text"
      value={searchQuery}      onChange={(e)
=> setSearchQuery(e.target.value)}
      placeholder="Search by name or roll number"
      />
      <select value={statusFilter} onChange={(e) => setStatusFilter(e.target.value)}>
        <option value="All">All Status</option>
        <option value="Present">Present</option>
        <option value="Absent">Absent</option>
      </select>
    </div>
  </div>
  {loading ? (
    <p>Loading...</p>
  ) : (
    <AttendanceList
records={attendanceRecords}
onUpdate={handleUpdate}
onDelete={handleDelete}
    />
  )}
</div>
</div>
</main>

```

```
</div>
```

```
); };
```

```
export default AttendancePortal;
```

## Login.jsx

```
import React, { useState } from 'react'; import
```

```
{ Link } from 'react-router-dom'; const Login
```

```
= ({ onLogin }) => {  const [email, setEmail]
```

```
= useState("");  const [password, setPassword]
```

```
= useState("");  const handleSubmit = async
```

```
(e) => {
```

```
  e.preventDefault();
```

```
  try {
```

```
    const response = await fetch('http://localhost:5000/api/login', {
```

```
      method: 'POST',
```

```
      headers: {
```

```
        'Content-Type':
```

```
        'application/json',
```

```
      },
```

```
      body: JSON.stringify({ email, password }),
```

```
    });    if (response.ok) {      const
```

```
userData = await response.json();
```

```
onLogin(userData);    } else {
```

```
  const error = await response.json();
```

```
alert(error.error || 'Login failed');
```

```
}
```

```
} catch (error) {
```

```
console.error('Login error:', error);
```

```
alert('Login failed. Please try again.');
```

```

    } };
return (
  <
    <header className="header">
      <div className="header-content">
        <h1>Students Attendance System</h1>
      </div>
    </header>
    <div className="login-container">
      <div className="login-card">
        <h2>Admin Login</h2>
        <form onSubmit={handleSubmit}>
          <div className="form-group">
            <label>Email:</label>
            <input
              type="email"          value={email}
              onChange={(e) => setEmail(e.target.value)}
              placeholder="admin@example.com"
              required
            />
          </div>
          <div className="form-group">
            <label>Password:</label>
            <input
              type="password"       value={password}
              onChange={(e) => setPassword(e.target.value)}
              placeholder="admin"
              required
            />
          </div>

```

```

        <button type="submit" className="btn btn-primary">Login</button>
    </form>

    <p>Don't have an account? <Link to="/register">Register</Link></p>
</div>
</div>
</>
); };
```

export default Login;

### **Register.jsx:**

```

import React, { useState } from 'react'; import
{ Link } from 'react-router-dom'; const
Register = () => {

    const [name, setName] = useState(""); const [email, setEmail]
= useState(""); const [password, setPassword] = useState("");
const [confirmPassword, setConfirmPassword] = useState("");
const handleSubmit = async (e) => {
    e.preventDefault();    if (password
!== confirmPassword) {
        alert('Passwords do not match');
        return;
    }    try {
        const response = await fetch('http://localhost:5000/api/register', {
method: 'POST',    headers: {
        'Content-Type': 'application/json',
    },
    body: JSON.stringify({ name, email, password }),
});
    if (response.ok) {        alert('Registration
successful! Please login. ');        setName("");
```



```

setEmail("");    setPassword("");
setConfirmPassword("");
    } else {
        const error = await response.json();
alert(error.error || 'Registration failed');
    }
    } catch (error) {
        console.error('Registration error:', error);
        alert('Registration failed. Please try
again.');
```

} };

return

(

◇

```

<header className="header">
    <div className="header-content">
        <h1>Students Attendance System</h1>
    </div>
</header>
<div className="register-container">
    <div className="login-card">
        <h2>Admin Registration</h2>
        <form onSubmit={handleSubmit}>
            <div className="form-group">
                <label>Name:</label>
                <input
                    type="text"
value={name}          onChange={(e) =>
setName(e.target.value)}
placeholder="Enter your name"

```

```
        required
    />

</div>

<div className="form-group">
    <label>Email:</label>

    <input      type="email"
value={email}      onChange={(e) =>
setEmail(e.target.value)}
placeholder="Enter your email"
```

```
        required
    />

</div>

<div className="form-group">
    <label>Password:</label>

    <input
        type="password"      value={password}
onChange={(e) => setPassword(e.target.value)}
placeholder="Enter password"
```

```
        required
    />

</div>

<div className="form-group">
    <label>Confirm Password:</label>

    <input
        type="password"      value={confirmPassword}
onChange={(e) => setConfirmPassword(e.target.value)}
placeholder="Confirm password"
```

```
        required
    />

</div>
```

```

        <button type="submit" className="btn btn-success">Register</button>
    </form>
    <p>Already have an account? <Link to="/login">Login</Link></p>
</div>
</div>
</>
);
};export default Register;

```

## Backend:

### Admin.js:

```

const mongoose = require('mongoose'); const
adminSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,    trim:
true
  },
  email: {    type:
String,
    required: true,
    unique: true,
    trim: true,
    lowercase: true
  },
  password: {
    type: String,
    required: true
  } },
{
  timestamps: true

```

```
});  
module.exports = mongoose.model('Admin', adminSchema);
```

### **Attendance.js:**

```
const mongoose = require('mongoose'); const  
attendanceSchema = new mongoose.Schema({  
  studentName: {  
    type: String,  
    required: true,    trim:  
    true  
  },  
  rollNumber: {  
    type: String,  
    required: true,  
    trim: true  },  
  date: {    type:  
    String,  
    required: true  
  },  
  status: {    type: String,  
    required: true,    enum:  
    ['Present', 'Absent']  
  } },  
  {  
    timestamps: true  
  });  
module.exports = mongoose.model('Attendance', attendanceSchema);
```

### **Server.js:**

```
const express = require('express'); const  
mongoose = require('mongoose');
```

```

const cors = require('cors'); const bodyParser =
require('body-parser'); const Admin =
require('./models/Admin'); const Attendance =
require('./models/Attendance'); const app =
express();

const PORT = process.env.PORT || 5000;
// MongoDB connection
const MONGODB_URI =
'mongodb+srv://vishnupreethi084_db_user:Admin%23123@cluster0.z24k1rk.mongodb.net/?appNa
me=Cluster0'; let useMongoDB = false; let admins = [
  { id: 1, name: 'Default Admin', email: 'admin@example.com', password: 'admin' },
];
let attendanceRecords = [
  { id: 1, studentName: 'John Doe', rollNumber: '101', date: '2023-05-01', status: 'Present' },
  { id: 2, studentName: 'Jane Smith', rollNumber: '102', date: '2023-05-01', status: 'Absent' },
  { id: 3, studentName: 'Bob Johnson', rollNumber: '103', date: '2023-05-01', status: 'Present' },
];
let nextAdminId = 2; let
nextAttendanceId = 4;
mongoose.connect(MONGODB_URI)
.then(() => { console.log('Connected to
MongoDB Atlas'); useMongoDB = true;
  // Seed initial data
  seedInitialData();
})
.catch((error) => { console.error('MongoDB connection failed, using in-memory
storage:', error.message); console.log('Server will run with in-memory data storage');
});
// Middleware app.use(cors());
app.use(bodyParser.json());

```

```

// Seed initial data
async function seedInitialData() {
  try {
    // Check if default admin exists
    const existingAdmin = await Admin.findOne({ email: 'admin@example.com' });
    if (!existingAdmin) {
      const
defaultAdmin = new Admin({
name: 'Default Admin',    email:
'admin@example.com',    password:
'admin'
    });
    await defaultAdmin.save();
    console.log('Default admin created');
  }

  // Check if attendance records exist    const attendanceCount
= await Attendance.countDocuments();    if (attendanceCount
=== 0) {    const initialAttendance = [
    { studentName: 'John Doe', rollNumber: '101', date: '2023-05-01', status: 'Present' },
    { studentName: 'Jane Smith', rollNumber: '102', date: '2023-05-01', status: 'Absent' },
    { studentName: 'Bob Johnson', rollNumber: '103', date: '2023-05-01', status: 'Present' },
  ];
    await Attendance.insertMany(initialAttendance);
    console.log('Initial attendance records created');
  }
} catch (error) {    console.error('Error
seeding data:', error);}

// Auth routes app.post('/api/login', async
(req, res) => { try {
    const { email, password } = req.body;

```

```

    if (useMongoDB) { const admin = await
    Admin.findOne({ email, password });
    if (admin) { res.json({ email: admin.email, name: admin.name,
role: 'admin' });
    } else { res.status(401).json({ error: 'Invalid
credentials' });
    }
    } else {
    const admin = admins.find(a => a.email === email && a.password === password);
    if (admin) { res.json({ email: admin.email, name: admin.name,
role: 'admin' });
    } else { res.status(401).json({ error: 'Invalid
credentials' });
    }
    }
  } catch (error) { console.error('Login error:', error);
res.status(500).json({ error: 'Internal server error' });
} }); app.post('/api/register', async (req,
res) => {
  try {
    const { name, email, password } = req.body; if (!name ||
!email || !password) { return res.status(400).json({ error: 'All
fields are required' });
    }
    if (useMongoDB) { const existingAdmin = await
    Admin.findOne({ email }); if (existingAdmin) { return
    res.status(400).json({ error: 'Email already registered' });
    }
  }

```

```

    const newAdmin = new Admin({ name, email, password });
    await newAdmin.save();    res.status(201).json({ message: 'Admin
registered successfully' });
  } else {
    const existingAdmin = admins.find(a => a.email === email);
    if (existingAdmin) {      return res.status(400).json({ error: 'Email
already registered' });
    }
    const newAdmin = {
id: nextAdminId++,
    name,
email,
password,
    };
    admins.push(newAdmin);    res.status(201).json({ message:
'Admin registered successfully' });
  }
} catch (error) {  console.error('Registration error:',
error);  res.status(500).json({ error: 'Internal server
error' });
}
});
// Attendance routes app.get('/api/attendance',
async (req, res) => {
  try {
    if (useMongoDB) {  const attendanceRecords = await
Attendance.find().sort({ createdAt: -1 });  res.json(attendanceRecords);
    } else {
      res.json(attendanceRecords);
    }
  }
}

```



```

    } catch (error) { console.error('Error fetching
attendance:', error); res.status(500).json({ error:
'Internal server error' });
    } });
app.get('/api/attendance/search', async (req, res) => {
  try {
    const { search, status } = req.query; const query = search
? search.toLowerCase() : ""; const statusFilter = status &&
status !== 'All' ? status : null; if (useMongoDB) {
      let searchQuery = {};
if (query) {
searchQuery = {
  $or: [
    { studentName: { $regex: query, $options: 'i' } },
    { rollNumber: { $regex: query, $options: 'i' } }
  ]
};
}
      if (statusFilter) {
searchQuery.status = statusFilter;
      }
const results = await Attendance.find(searchQuery).sort({ createdAt: -1 });res.json(results);
    } else {
      let results = attendanceRecords;
      if (query) { results =
results.filter(record =>
record.studentName.toLowerCa
se().includes(query) ||
record.rollNumber.toLowerCase
().includes(query)

```

```

    );
    } if (statusFilter) { results = results.filter(record =>
record.status === statusFilter);
    }
    res.json(results);
  }
} catch (error) { console.error('Error searching
attendance:', error); res.status(500).json({ error:
'Internal server error' });
} });
app.post('/api/attendance', async (req, res) => {
  try {
    const { studentName, rollNumber, date, status } = req.body;
    if (!studentName || !rollNumber || !date || !status) { return
res.status(400).json({ error: 'Missing required fields' });
    }
    if (useMongoDB) { const newRecord = new Attendance({ studentName,
rollNumber, date, status }); await newRecord.save();
res.status(201).json(newRecord);
    } else {
      const newRecord = {
        id: nextAttendanceId++,
        studentName,
        rollNumber, date,
        status,
      };
      attendanceRecords.push(newRecord);
      res.status(201).json(newRecord);
    }
  }
}

```

```

    } catch (error) { console.error('Error adding
attendance:', error); res.status(500).json({ error:
'Internal server error' }); } });
app.put('/api/attendance/:id', async (req, res) => {
  try {
    const { status } = req.body;
    if (!status) { return res.status(400).json({ error:
'Status is required' }); }
    if (useMongoDB) { const updatedRecord = await
Attendance.findByIdAndUpdate(
      req.params.id,
      { status },
      { new: true, runValidators: true }
    );
    if (!updatedRecord) { return res.status(404).json({
error: 'Record not found' }); }
    res.json(updatedRecord);
  } else {
const recordIndex = attendanceRecords.findIndex(record => record.id === parseInt(req.params.id));
    if (recordIndex === -1) { return res.status(404).json({ error: 'Record not found' }); }
    attendanceRecords[recordIndex].status = status;
    res.json(attendanceRecords[recordIndex]);
  }
} catch (error) { console.error('Error updating
attendance:', error); res.status(500).json({ error:
'Internal server error' }); } });
app.delete('/api/attendance/:id', async (req,
res) => {

```

```

try {
  if (useMongoDB) {    const deletedRecord = await
Attendance.findByIdAndDelete(req.params.id);
    if (!deletedRecord) {    return res.status(404).json({
error: 'Record not found' });
    }
    res.json(deletedRecord);
  } else {
    const recordIndex = attendanceRecords.findIndex(record => record.id ===
parseInt(req.params.id));    if (recordIndex === -1) {    return
res.status(404).json({ error: 'Record not found' });
    }
    const deletedRecord = attendanceRecords.splice(recordIndex, 1)[0];
res.json(deletedRecord);
  }
} catch (error) {console.error('Error deleting attendance:', error);
res.status(500).json({ error: 'Internal server error' });
}
});app.listen(PORT, () => {  console.log(`Server is
running on port ${PORT}`);});

```

OUTPUT:

Students Attendance System

Admin Login

Email:

admin@example.com

Password:

admin

Login

Don't have an account? [Register](#)

Students Attendance System

Admin Registration

Name:

Enter your name

Email:

Enter your email

Password:

Enter password

Confirm Password:

Confirm password

Register

Already have an account? [Login](#)

Students Attendance Portal

Welcome, Admin (Admin@gmail.com) [Logout](#)

Add New Attendance Record

Student Name:

Enter student name

Roll Number:

Enter roll number

Date:

dd-mm-yyyy

Status:

Present

Add Record

Attendance Records

Search by name or roll number

All Status

| Student Name | Roll Number | Date       | Status  | Actions                                     |
|--------------|-------------|------------|---------|---------------------------------------------|
| John Doe     | 101         | 2023-05-01 | Present | <a href="#">Edit</a> <a href="#">Delete</a> |
| Jane Smith   | 102         | 2023-05-01 | Absent  | <a href="#">Edit</a> <a href="#">Delete</a> |
| Bob Johnson  | 103         | 2023-05-01 | Present | <a href="#">Edit</a> <a href="#">Delete</a> |

Students Attendance Portal

Welcome, Admin (Admin@gmail.com)Logout

Add New Attendance Record

Student Name:

Enter student name

Roll Number:

Enter roll number

Date:

dd-mm-yyyy

Status:

Present

Add Record

Attendance Records

Search by name or roll number

All Status

| Student Name  | Roll Number | Date       | Status  | Actions               |
|---------------|-------------|------------|---------|-----------------------|
| John Doe      | 101         | 2023-05-01 | Present | <div>EditDelete</div> |
| Jane Smith    | 102         | 2023-05-01 | Absent  | <div>EditDelete</div> |
| Bob Johnson   | 103         | 2023-05-01 | Present | <div>EditDelete</div> |
| Vishnupreethi | 350         | 2026-05-04 | Present | <div>EditDelete</div> |

Students Attendance Portal

Welcome, Admin (Admin@gmail.com)Logout

Add New Attendance Record

Student Name:

Enter student name

Roll Number:

Enter roll number

Date:

dd-mm-yyyy

Status:

Present

Add Record

Attendance Records

Search by name or roll number

Present

| Student Name  | Roll Number | Date       | Status  | Actions               |
|---------------|-------------|------------|---------|-----------------------|
| John Doe      | 101         | 2023-05-01 | Present | <div>EditDelete</div> |
| Bob Johnson   | 103         | 2023-05-01 | Present | <div>EditDelete</div> |
| Vishnupreethi | 350         | 2026-05-04 | Present | <div>EditDelete</div> |

Students Attendance Portal

Welcome, Admin (Admin@gmail.com)Logout

Add New Attendance Record

Student Name:

Enter student name

Roll Number:

Enter roll number

Date:

dd-mm-yyyy

Status:

Present

Add Record

Attendance Records

Search by name or roll number

Present

| Student Name  | Roll Number | Date       | Status  | Actions               |
|---------------|-------------|------------|---------|-----------------------|
| John Doe      | 101         | 2023-05-01 | Present | <div>EditDelete</div> |
| Jane Smith    | 102         | 2023-05-01 | Absent  | <div>EditDelete</div> |
| Bob Johnson   | 103         | 2023-05-01 | Absent  | <div>EditDelete</div> |
| Vishnupreethi | 350         | 2026-05-04 | Present | <div>EditDelete</div> |

# POSTMAN OUTPUT:

Postman interface showing a GET request to `http://localhost:5000/api/attendance`. The response is a 200 OK status with a 280 ms response time and 550 B of data. The response body is a JSON array of three attendance records.

| Key | Value | Description |
|-----|-------|-------------|
| Key | Value | Description |

```
1 [
2   {
3     "id": 1,
4     "studentName": "John Doe",
5     "rollNumber": "101",
6     "date": "2023-05-01",
7     "status": "Present"
8   },
9   {
10    "id": 2,
11    "studentName": "Jane Smith",
12    "rollNumber": "102",
13    "date": "2023-05-01",
14    "status": "Absent"
15  },
16  {
17    "id": 3,
18    "studentName": "Bob Johnson",
19    "rollNumber": "103",
20    "date": "2023-05-01",
21    "status": "Present"
22  }
23 ]
```

Postman interface showing a GET request to `http://localhost:5000/api/attendance/search?search=John&status=Present`. The response is a 200 OK status with a 51 ms response time and 456 B of data. The response body is a JSON array of two attendance records.

| Key | Value | Description |
|-----|-------|-------------|
| Key | Value | Description |

```
1 [
2   {
3     "id": 1,
4     "studentName": "John Doe",
5     "rollNumber": "101",
6     "date": "2023-05-01",
7     "status": "Present"
8   },
9   {
10    "id": 3,
11    "studentName": "Bob Johnson",
12    "rollNumber": "103",
13    "date": "2023-05-01",
14    "status": "Present"
15  }
16 ]
```

Postman interface showing a POST request to `http://localhost:5000/api/attendance`. The request body is a JSON object representing a new attendance record. The response is a 201 Created status with a 10 ms response time and 371 B of data. The response body is a JSON object representing the created attendance record.

```
1 {
2   "studentName": "Alice Brown",
3   "rollNumber": "24CSR104",
4   "date": "2023-05-02",
5   "status": "Present"
6 }
```

```
1 {
2   "id": 4,
3   "studentName": "Alice Brown",
4   "rollNumber": "24CSR104",
5   "date": "2023-05-02",
6   "status": "Present"
7 }
```

PUT http://localhost:5000/api/attendance/4

Send

Body

```
1 {
2   "status": "Absent"
3 }
```

Body

```
1 {
2   "id": 4,
3   "studentName": "Alice Brown",
4   "rollNumber": "24CSE104",
5   "date": "2023-05-02",
6   "status": "Absent"
7 }
```

200 OK · 63 ms · 365 B

DELETE http://localhost:5000/api/attendance/2

Send

Params

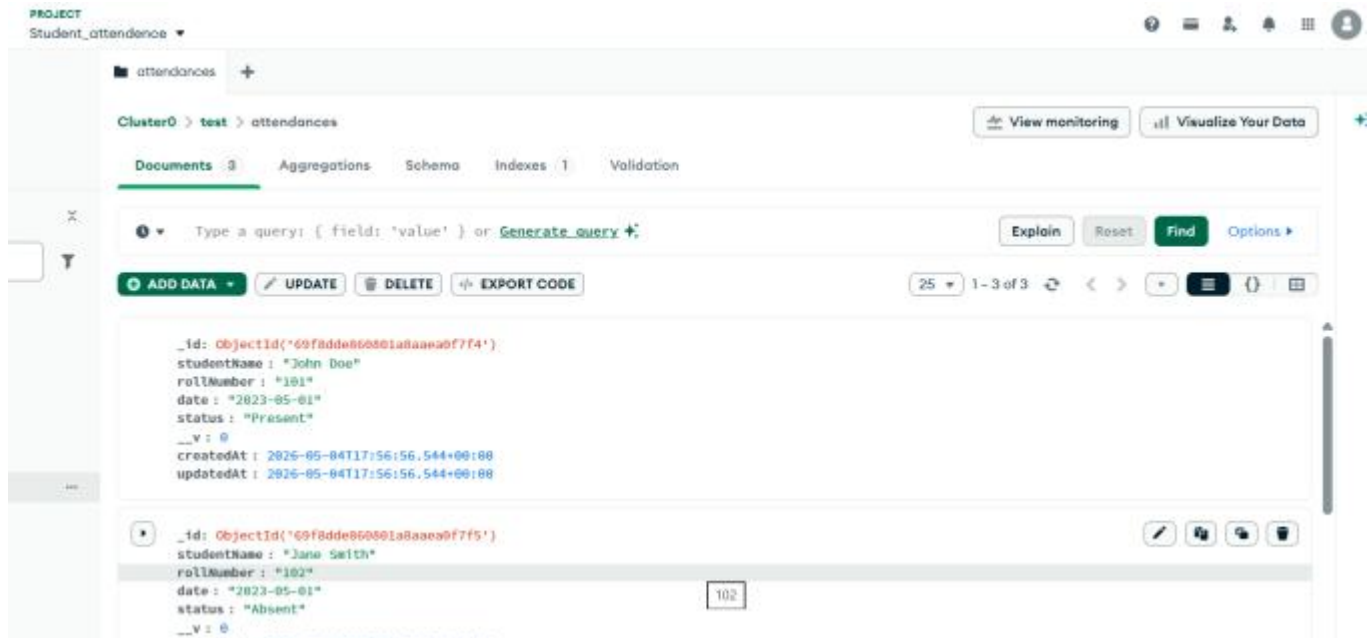
| Key | Value | Description |
|-----|-------|-------------|
| Key | Value | Description |

Body

```
1 {
2   "error": "Record not found"
3 }
```

404 Not Found · 8 ms · 302 B





## CONCLUSION:

The Student Attendance Portal successfully demonstrates how modern web technologies can be integrated to build an efficient and user-friendly application for managing attendance. By using React.js for the frontend, Node.js and Express.js for the backend, and MongoDB for data storage, the system performs essential operations such as fetching, updating, and deleting attendance records through REST APIs. The application provides real-time updates without page refresh, improving usability and performance. Overall, this project highlights the practical implementation of CRUD operations and serves as a scalable solution for digital attendance management in educational institutions.